

Designing & Solving Real World Synchronization Problems



CSE 4501 : Operating Systems

Assigned to :

Saadman Sakib

ID: 210042165

Program : BSc. Engg. SWE

Department of Computer Science and Engineering

Islamic University of Technology

Design and Resolution of Real-World Synchronization Problems Using OS Techniques

Abstract :

This assignment focuses on the design and resolution of real-world synchronization problems using OS-level techniques. Three original problems are proposed. Each of them reflects common scenarios in modern software/ hardware systems, where concurrent access to shared resources may lead to race conditions, deadlocks, or starvation. Each problem stages critical synchronization challenges and the solutions explains how appropriate mechanisms like semaphores, mutexes, monitors, or deadlock avoidance strategies can ensure safe and predictable execution.

1. Problem : 01

1.1 Problem Statement

A multi-level parking garage allows cars to enter and exit at the same time. Each level has a fixed number of slots. Cars entering must check for available slots and park; cars exiting must free up a slot. If multiple cars try to enter at the same time when only one slot is available, synchronization is needed to avoid overbooking.

1.2 Challenges Identified

Race condition: Two cars might be assigned the same last available slot. (Will cars 'race' for the last slot?)

Starvation: Entry processes could starve if exits are prioritized. (A car cannot enter for a long time)

Deadlock: Could occur if entry waits on space, but exit cannot proceed due to resource lock on the status check.

1.3 Solution and Justification:

We need to use a **counting semaphore** to represent available slots. Each entry thread must `wait()` on the semaphore before entering; exiting cars `signal()` the semaphore upon departure.

- Protect slot assignment with a **mutex** to avoid two cars taking the same slot.
- Fair access can be managed using **FIFO queueing** or semaphores with fair scheduling.

Justification:

Semaphores handle concurrent access well and are ideal for managing a limited resource (slots). Mutex ensures mutual exclusion during assignment.

1.4 Solution Code :

```
#include <semaphore.h>
#include <mutex>
#include <thread>
#include <stdio.h>

sem_t slots;
// Counting semaphore for parking slots
mutex slot_mutex;
// Protect slot assignment

void car_enter()
{
    sem_wait(&slots);
    // Wait for an available slot
    lock_guard<mutex> lock(slot_mutex);
    // Assign slot safely
    cout << "Car parked\n";
}

void car_exit()
{
    lock_guard<mutex> lock(slot_mutex);
    // Free up the slot
    cout << "Car left\n";
    sem_post(&slots);
    // Release a slot
}
```

2. Problem : 02

2.1 Problem Statement

In IUT Library, multiple students, faculty members and staff send print jobs to a shared printer. However, some users (e.g., faculty members) are allowed to send high-priority jobs that must interrupt normal ones. Still, a job already being printed must finish first.

2.2 Challenges Identified

Starvation: Normal users could be indefinitely delayed by a stream of high priority jobs.

Race condition: Multiple jobs sent at the same time may conflict in priority handling.

Fairness: Needs to balance quality of service with access fairness.

2.3 Solution and Justification:

We need to use monitors with condition variables for two queues:

- `normalQueue`
- `priorityQueue`

A print server thread checks the priority queue first, then serves the normal queue. We can use a mutex lock for enqueue/dequeue and condition variables to wait/signal threads.

Justification:

Monitors provide built-in mutual exclusion and condition synchronization, so they are ideal for coordinating resource access based on priority. Starvation can be addressed with a fairness counter or ratio-based priority serving (e.g., 3 priority jobs, then 1 normal job).

2.4 Solution Code :

```
#include <mutex>
#include <condition_variable>
#include <queue>
#include <stdio.h>

mutex mtx;
condition_variable cv;
queue<int> normalQueue, priorityQueue;

void submit_job(bool is_priority, int job_id)
{
    unique_lock<mutex> lock(mtx);
    if (is_priority) priorityQueue.push(job_id);
    else normalQueue.push(job_id);
    cv.notify_one();
}

void printer()
{
    while (true) {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, [] {
            return !priorityQueue.empty() ||
!normalQueue.empty();
        });
    }
}
```

```
        if (!priorityQueue.empty())
        {
            int job = priorityQueue.front();
priorityQueue.pop();
            cout << "Printing priority job " <<
job << "\n";
        }
        else
        {
            int job = normalQueue.front();
normalQueue.pop();
            cout << "Printing normal job " <<
job << "\n";
        }
    }
}
```

3. Problem : 03

3.1 Problem Statement

Multiple robot arms in a warehouse pick items from shared storage bins and deliver them to packaging units. The same item type may be stored in multiple bins. Robots must coordinate to avoid picking from the same bin at the same time (to prevent bin overload or collisions), and update the inventory count atomically.

3.2 Challenges Identified

Race condition: Two robots may read the same item count and try to pick at once.

Data inconsistency: Without synchronization, stock levels may become corrupted.

Deadlock: If multiple robots lock multiple bins in conflicting order.

3.3 Solution and Justification:

We need to use a mutex per bin to ensure exclusive access to the bin while picking.

To prevent deadlock during multiple bin access:

- We have to enforce a global bin lock ordering (always acquire in order of bin ID).

Then we have to use atomic operations / semaphores to handle inventory updates safely.

Justification:

Per-resource locking minimizes contention while still ensuring safety. Ordering prevents circular wait, eliminating deadlock. This is a common pattern in database and file system design as well.

3.4 Solution Code :

```
#include <mutex>
#include <vector>
#include <algorithm>
#include <stdio.h>

vector<mutex> bin_mutexes(3); //Let's assume 3 bins

void pick_items_from_bins(vector<int> bins)
{
    sort(bins.begin(), bins.end());
    // Enforce Lock order
    for (int b : bins) bin_mutexes[b].lock();

    // Safely pick items from bins
    cout << "Robot picking from bins\n";

    for (int i = bins.size() - 1; i >= 0; --i)
        bin_mutexes[bins[i]].unlock();
}
```